STA 9750 Mini-Project 01

STA 9750 Mini-Project 01 - Akshan Rai

1. Data Acquisition

```r
source("https://michael-weylandt.com/STA9750/load_helpers.R")
#mp_start(N=01)
load_netflix_data <- function(){
  # Note to students: Make sure you have installed the tidyverse and fs packages
  #                   before beginning this assignment. If you aren't sure,
  #                   simply run `library(tidyverse)` at your R Console and, if
  #                   you don't see an error message, `tidyverse` has been
  #                   installed. (Same for `fs`) If you still need to install
  #                   either package, use the `install.packages` function as
  #                   demonstrated in class.
  library(tidyverse)
  library(rvest)
  library(readxl)
  library(fs)

  data_dir <- path("data", "mp01")

  if(!dir_exists(data_dir)){
    dir_create(data_dir, showWarnings=FALSE, recurse=TRUE)
  }

  NETFLIX_URLS <- list(
    `2023H1` = "https://about.netflix.com/news/what-we-watched-a-netflix-engagement-repo
    `2023H2` = "https://about.netflix.com/news/what-we-watched-the-second-half-of-2023",
    `2024H1` = "https://about.netflix.com/news/what-we-watched-the-first-half-of-2024",
    `2024H2` = "https://about.netflix.com/news/what-we-watched-the-second-half-of-2024",
    `2025H1` = "https://about.netflix.com/news/what-we-watched-the-first-half-of-2025",
    `2025H2` = "https://about.netflix.com/news/what-we-watched-the-second-half-of-2025",
    `2026H1` = "https://about.netflix.com/news/what-we-watched-the-first-half-of-2026"
  )

  download_if_needed <- function(url, local){
    if(!file_exists(local)){
      download.file(url, local, quiet = TRUE, mode = "wb")
    }

    return(local)
  }

  NETFLIX_URLS |>
    map(read_html) |>
    map(html_element, xpath = "//a[contains(@href, '.xlsx')]") |>
    map_chr(html_attr, "href") |>
    enframe("period", "url") |>
    mutate(path = path(data_dir, paste0(period, ".xlsx")),
           path = map2(url, path, download_if_needed),
           data = map(path, read_excel, skip=3, na="*"),
           data = map(data, \(x) x |> mutate(`Release Date` = as.Date(`Release Date`))))
    unnest(data) |>
    select(-url, -path) |>
    mutate(year=str_sub(period, 1, 4),
           half=str_sub(period, start=6),
           period_start = make_date(year, if_else(half==1, 1, 7), 1),
           period_end = make_date(year, if_else(half==1, 6, 12), if_else(half==1, 30, 31
    filter(!is.na(`Hours Viewed`)) |>
    mutate(live_event = !is.na(Runtime) & str_detect(Runtime, "\\d+:\\d+\\*"),
           clean_time = str_remove(Runtime, fixed("*")),
           Runtime = 60 * hour(hm(clean_time)) + minute(hm(clean_time))) |>
    select(-clean_time) |>
    mutate(Runtime = coalesce(Runtime, mean(Runtime, na.rm=TRUE)),
           .by=Title,
           Title = stringi::stri_trans_general(Title, "Latin-ASCII"),
           Title = str_remove(Title, " // .*"),
           Title = str_remove(Title, "[^[:ascii:]].*"),
           Title = str_trim(Title)) |>
    filter_out(Title == "") |>
    select(year,
           period,
           period_start,
           period_end,
           Title,
           -half,
           Runtime,
           Views,
           `Hours Viewed`,
           `Release Date`,
           `Available Globally?`,
           live_event)
}

NETFLIX_DATA <- load_netflix_data()
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✔ dplyr     1.2.1     ✔ readr     2.2.0
✔ forcats   1.0.1     ✔ stringr   1.6.0
✔ ggplot2   4.0.3     ✔ tibble    3.3.1
✔ lubridate 1.9.5     ✔ tidyr     1.3.2
✔ purrr     1.2.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to b

Attaching package: 'rvest'


The following object is masked from 'package:readr':

    guess_encoding
```

```r
NETFLIX_DATA
```

```
# A tibble: 61,892 × 11
   year  period period_start period_end Title       Runtime Views `Hours Viewed`
   <chr> <chr>  <date>       <date>     <chr>         <dbl> <dbl>          <dbl>
 1 2023  2023H1 2023-01-01   2023-06-30 The Night …     491    NA      812100000
 2 2023  2023H1 2023-01-01   2023-06-30 Ginny & Ge…     587    NA      665100000
 3 2023  2023H1 2023-01-01   2023-06-30 The Glory:…     856    NA      622800000
 4 2023  2023H1 2023-01-01   2023-06-30 Wednesday:…     409    NA      507700000
 5 2023  2023H1 2023-01-01   2023-06-30 Queen Char…     389    NA      503000000
 6 2023  2023H1 2023-01-01   2023-06-30 You: Seaso…     497    NA      440600000
 7 2023  2023H1 2023-01-01   2023-06-30 La Reina d…    2703    NA      429600000
 8 2023  2023H1 2023-01-01   2023-06-30 Outer Bank…     533    NA      402500000
 9 2023  2023H1 2023-01-01   2023-06-30 Ginny & Ge…     548    NA      302100000
10 2023  2023H1 2023-01-01   2023-06-30 FUBAR: Sea…     428    NA      266200000
# ℹ 61,882 more rows
# ℹ 3 more variables: `Release Date` <date>, `Available Globally?` <chr>,
#   live_event <lgl>
```

2. Fix Column Names and Types

```r
NETFLIX_DATA <- NETFLIX_DATA |>
  rename(
    title = Title,
    runtime = Runtime,
    views = Views,
    hours_viewed = `Hours Viewed`,
    release_date = `Release Date`,
    available_globally = `Available Globally?`
  )

NETFLIX_DATA <- NETFLIX_DATA |>
  mutate(
    year = as.integer(year),
    available_globally = available_globally == "Yes"
  )
```

3. Exploratory Data Analysis

```r
glimpse(NETFLIX_DATA)
```

```
Rows: 61,892
Columns: 11
$ year               <int> 2023, 2023, 2023, 2023, 2023, 2023, 2023, 2023, 202…
$ period             <chr> "2023H1", "2023H1", "2023H1", "2023H1", "2023H1", "…
$ period_start       <date> 2023-01-01, 2023-01-01, 2023-01-01, 2023-01-01, 20…
$ period_end         <date> 2023-06-30, 2023-06-30, 2023-06-30, 2023-06-30, 20…
$ title              <chr> "The Night Agent: Season 1", "Ginny & Georgia: Seas…
$ runtime            <dbl> 491, 587, 856, 409, 389, 497, 2703, 533, 548, 428, …
$ views              <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,…
$ hours_viewed       <dbl> 812100000, 665100000, 622800000, 507700000, 5030000…
$ release_date       <date> 2023-03-23, 2023-01-05, 2022-12-30, 2022-11-23, 20…
$ available_globally <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, FALSE, TRUE, TR…
$ live_event         <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA…
```

```r
summary(NETFLIX_DATA)
```

```
      year           period           period_start          period_end
 Min.   :2023   Length    :61892   Min.   :2023-01-01   Min.   :2023-06-30
 1st Qu.:2023   N.unique :    7   1st Qu.:2023-01-01   1st Qu.:2023-06-30
 Median :2024   N.blank  :    0   Median :2024-01-01   Median :2024-06-30
 Mean   :2024   Min.nchar:    6   Mean   :2024-04-12   Mean   :2024-10-10
 3rd Qu.:2025   Max.nchar:    6   3rd Qu.:2025-07-01   3rd Qu.:2025-12-31
 Max.   :2026                     Max.   :2026-01-01   Max.   :2026-06-30

      title           runtime            views             hours_viewed
 Length   :61892   Min.   :    2.0   Min.   :   100000   Min.   :   100000
 N.unique :24427   1st Qu.:  210.0   1st Qu.:   100000   1st Qu.:   500000
 N.blank  :    0   Median :  320.0   Median :   400000   Median :  1800000
 Min.nchar:    1   Mean   :  467.3   Mean   :  1375241   Mean   :  8265858
 Max.nchar:  217   3rd Qu.:  545.0   3rd Qu.:  1200000   3rd Qu.:  6400000
                   Max.   :23853.0   Max.   :144800000   Max.   :964100000
                   NAs    :13085     NAs    :18198
  release_date        available_globally live_event
 Min.   :2010-04-01   Mode :logical      Mode :logical
 1st Qu.:2019-05-31   FALSE:40171        FALSE:61582
 Median :2021-06-17   TRUE :21719        TRUE :310
 Mean   :2021-03-25   NAs  :2
 3rd Qu.:2023-02-14
 Max.   :2026-06-29
 NAs    :35251
```

```r
#install.packages("skimr")
library(skimr)
skim(NETFLIX_DATA)
```

Data summary

Name	NETFLIX_DATA
Number of rows	61892
Number of columns	11
_______________________	
Column type frequency:	
character	2
Date	3
logical	2
numeric	4
________________________	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
period	0	1	6	6	0	7	0
title	0	1	1	217	0	24427	0

Variable type: Date

skim_variable	n_missing	complete_rate	min	max	median	n_unique
period_start	0	1.00	2023-01-01	2026-01-01	2024-01-01	7
period_end	0	1.00	2023-06-30	2026-06-30	2024-06-30	7
release_date	35251	0.43	2010-04-01	2026-06-29	2021-06-17	2487

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
available_globally	2	1	0.35	FAL: 40171, TRU: 21719
live_event	0	1	0.01	FAL: 61582, TRU: 310

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
year	0	1.00	2024.11	1.08	2023	2023	2024	2025	2026
runtime	13085	0.79	467.35	694.71	2	210	320	545	23853
views	18198	0.71	1375241.45	3630933.22	100000	100000	400000	1200000	144800000
hours_viewed	0	1.00	8265858.27	24924226.03	100000	500000	1800000	6400000	964100000

4. Exploratory Questions - Inline Values

Question 1: What date range is covered by this data set?

The data covers the period from January 1, 2023 through June 30, 2026

```r
#1. What date range is covered by this data set?
NETFLIX_DATA |>
  summarize(
    start_date = min(period_start),
    end_date = max(period_end)
  ) #The data covers the period from January 1, 2023 through June 30, 2026
```

```
# A tibble: 1 × 2
  start_date end_date
  <date>     <date>
1 2023-01-01 2026-06-30
```

Question 2: How many different shows and movies are covered in this data set?

there are 24,427 different shows/movies in this data set

```r
#2. How many different shows and movies are covered in this data set?
NETFLIX_DATA |>
  summarize(
    n_titles = n_distinct(title)
  ) #If you include different seasons of shows, there are 24,427 different shows/movies
```

```
# A tibble: 1 × 1
  n_titles
     <int>
1    24427
```

Question 3: How many total hours have been spent watching shows from the Bridgerton universe?

4,862,800,000 hours have been spent watching the shows from the Bridgerton universe.

```r
#3. How many total hours have been spent watching shows from the Bridgerton universe?

NETFLIX_DATA |>
  filter(str_detect(title, "Bridgerton")) |> #includes Bridgerton and Queen Charlotte
  summarize(
    bridgerton_total_hours = sum(hours_viewed)
  ) #4,862,800,000 hours have been spent watching the shows from the Bridgerton universe
```

```
# A tibble: 1 × 1
  bridgerton_total_hours
                   <dbl>
1             4862800000
```

Question 4: How many total hours have been spent watching the five seasons of Stranger Things?

5,548,500,000 hours have been spent watchning the five seasons of Stranger Things.

```r
#4. How many total hours have been spent watching the five seasons of Stranger Things?

NETFLIX_DATA |>
  filter(str_detect(title, "Stranger Things")) |>
  distinct(title) #First find the actual names of seasons 1-5
```

```
# A tibble: 8 × 1
  title
  <chr>
1 Stranger Things 4
2 Stranger Things
3 Stranger Things 2
4 Stranger Things 3
5 Beyond Stranger Things: Beyond 2
6 Stranger Things 5
7 Stranger Things: Fireplace
8 Stranger Things: Tales From '85: Season 1
```

```r
NETFLIX_DATA |>
  filter(str_detect(title, "Stranger Things")) |>
  filter_out(str_detect(title, "Beyond Stranger Things")) |>
  filter_out(str_detect(title, "Tales From '85")) |>
  filter_out(str_detect(title, "Fireplace")) |>
  summarize(
    st_total_hours = sum(hours_viewed)
  ) #5,548,500,000 hours have been spent watchning the five seasons of Stranger Things.
```

```
# A tibble: 1 × 1
  st_total_hours
           <dbl>
1     5548500000
```

Question 5: What percentage of total watch time was spent on the Top 10 titles of 2026H1?

The top 10 titles of the first half of 2026 accounted for 6.6967% of the total hours watched.

```r
#5. What percentage of total watch time was spent on the Top 10 titles of 2026H1?

NETFLIX_DATA |>
  filter(period == "2026H1") |>
  arrange(desc(hours_viewed)) |>
  slice_head(n = 10) |>
  summarize(
    top_10_titles_hours = sum(hours_viewed) #this give us the total hours for the top 10
  ) #4,977,900,000 hours
```

```
# A tibble: 1 × 1
  top_10_titles_hours
                <dbl>
1          4977900000
```

```r
NETFLIX_DATA |>
  filter(period == "2026H1") |>
  summarize(
    total_hours = sum(hours_viewed) #total hours watched for 2026H1
  ) #74,333,600,000 total hours watched
```

```
# A tibble: 1 × 1
  total_hours
        <dbl>
1 74333600000
```

```r
pct_top_ten = (4977900000 / 74333600000) * 100
pct_top_ten #6.6967%
```

```
[1] 6.696702
```

```r
#The top 10 titles of the first half of 2026 accounted for 6.6967% of the total hours wa
```

Question 6: What fraction of Netflix's catalog is globally available?

35.1% of Netflix's catalog is globally available.

```r
#6. What fraction of Netflix's catalog is globally available? (Answer this both by title

NETFLIX_DATA |>
  filter(!is.na(available_globally)) |> #this is to filter out the two NaN values we hav
  summarize(
    global_fraction = mean(available_globally) #Calculates the fraction (1=True, 0=False
  ) #35.1% of Netflix's catalog is globally available.
```

```
# A tibble: 1 × 1
  global_fraction
            <dbl>
1           0.351
```

5. Exploratory Questions - Table Answers

Question 1: What was the top title in each period?

```r
#1.What was the top title (by total hours viewed) in each period and what fraction of to

#install.packages("gt")
library(gt)

period_totals <- NETFLIX_DATA |>
  group_by(period) |>
  summarize(
    total_hours = sum(hours_viewed) #helps us find total hours per period
  )
top_titles <- NETFLIX_DATA |>
  group_by(period) |>
  summarize(
    title = title[which.max(hours_viewed)],
    hours_viewed = max(hours_viewed) #helps us find maximum hours viewed per period.
  ) #
top_titles
```

```
# A tibble: 7 × 3
  period title                          hours_viewed
  <chr>  <chr>                                 <dbl>
1 2023H1 The Night Agent: Season 1         812100000
2 2023H2 King the Land: Limited Series     630200000
3 2024H1 Bridgerton: Season 3              733800000
4 2024H2 Squid Game: Season 2              619900000
5 2025H1 Squid Game: Season 2              840300000
6 2025H2 Wednesday: Season 2               964100000
7 2026H1 Bridgerton: Season 4              889800000
```

```r
top_titles <- top_titles |>
  left_join(period_totals, by = "period") |>
  mutate(
    fraction = hours_viewed / total_hours
  )
top_titles
```

```
# A tibble: 7 × 5
  period title                          hours_viewed total_hours fraction
  <chr>  <chr>                                 <dbl>       <dbl>    <dbl>
1 2023H1 The Night Agent: Season 1         812100000 93407600000  0.00869
2 2023H2 King the Land: Limited Series     630200000 64931000000  0.00971
3 2024H1 Bridgerton: Season 3              733800000 68244300000  0.0108
4 2024H2 Squid Game: Season 2              619900000 68179900000  0.00909
5 2025H1 Squid Game: Season 2              840300000 71110800000  0.0118
6 2025H2 Wednesday: Season 2               964100000 71383300000  0.0135
7 2026H1 Bridgerton: Season 4              889800000 74333600000  0.0120
```

```r
top_titles |>
  select(period, title, fraction) |>
  gt() |>
  cols_label(
    period = "Period",
    title = "Top Title",
    fraction = "Share of Viewership"
  ) |>
  fmt_percent(
    fraction,
    decimals = 1
  ) |>
  tab_header(
    title = md("**Top Netflix Title by Viewing Period**"),
    subtitle = "Share of total watch time captured by the most-watched title"
  )
```

Top Netflix Title by Viewing Period

Share of total watch time captured by the most-watched title

Period	Top Title	Share of Viewership
2023H1	The Night Agent: Season 1	0.9%
2023H2	King the Land: Limited Series	1.0%
2024H1	Bridgerton: Season 3	1.1%
2024H2	Squid Game: Season 2	0.9%
2025H1	Squid Game: Season 2	1.2%
2025H2	Wednesday: Season 2	1.4%
2026H1	Bridgerton: Season 4	1.2%

Question 2: How has Netflix's catalog changed in size?

```r
#2. How has Netflix's catalog changed in size over the periods for which we have data? Y

catalog_size <- NETFLIX_DATA |>
  group_by(period) |>
  summarize(
    n_titles = n_distinct(title),
    total_runtime = sum(runtime, na.rm = TRUE)
  )

catalog_size |>
  gt() |>
  cols_label(
    period = "Period",
    n_titles = "Number of Titles",
    total_runtime = md("Total Runtime (*minutes*)")
  ) |>
  fmt_number(
    n_titles,
    decimals = 0
  ) |>
  fmt_number(
    total_runtime,
    decimals = 0
  ) |>
  tab_header(
    title = md("**Netflix Catalog Size Over Time**"),
    subtitle = "Number of titles and total runtime by reporting period"
  )
```

Netflix Catalog Size Over Time

Number of titles and total runtime by reporting period

Period	Number of Titles	Total Runtime (*minutes*)
2023H1	18,126	2,623,021
2023H2	6,574	3,007,003
2024H1	6,775	3,106,470
2024H2	6,854	3,216,885
2025H1	7,473	3,502,435
2025H2	7,712	3,582,841
2026H1	8,163	3,771,293

Question 3: Which titles have had the most consistently high viewership?

```r
#3. Which titles have had the most consistently high viewership across all periods? Thes

consistent_titles <- NETFLIX_DATA |>
  filter(!is.na(views)) |>
  group_by(title) |>
  summarize(
    avg_views = mean(views),
    min_views = min(views),
    sd_views = sd(views),
    n_periods = n_distinct(period)
  )

consistent_titles |>
  count(n_periods)
```

```
# A tibble: 6 × 2
  n_periods     n
      <int> <int>
1         1  2884
2         2  2060
3         3  1299
4         4  1085
5         5   858
6         6  4003
```

```r
consistent_titles |>
  arrange(desc(avg_views))
```

```
# A tibble: 12,189 × 5
   title                           avg_views min_views  sd_views n_periods
   <chr>                               <dbl>     <dbl>     <dbl>     <int>
 1 His & Hers: Limited Series      104000000 104000000       NA          1
 2 Bridgerton: Season 4            100200000 100200000       NA          1
 3 Stranger Things 5                74550000  55600000 26799347.         2
 4 Wednesday: Season 2              66550000   9200000 81105148.         2
 5 I Will Find You: Limited Series  63900000  63900000       NA          1
 6 Adolescence: Limited Series      57200000   6700000 76159110.         3
 7 Squid Game: Season 2             56350000   4500000 54322831.         4
 8 Squid Game: Season 3             51933333.  5300000 40559380.         3
 9 UNTAMED: Season 1                50550000   8300000 59750523.         2
10 Run Away: Limited Series         50300000  50300000       NA          1
# ℹ 12,179 more rows
```

```r
consistent_titles |>
  filter(n_periods >= 3) |>
  arrange(desc(avg_views))
```

```
# A tibble: 7,245 × 5
   title                       avg_views min_views  sd_views n_periods
   <chr>                           <dbl>     <dbl>     <dbl>     <int>
 1 Adolescence: Limited Series  57200000    6700000 76159110.         3
 2 Squid Game: Season 2         56350000    4500000 54322831.         4
 3 Squid Game: Season 3         51933333.   5300000 40559380.         3
 4 Ms. Rachel: Season 1         45833333.  37300000  8093413.         3
 5 Bridgerton: Season 3         30840000    7700000 34830274.         5
 6 Fool Me Once: Limited Series 27660000    2800000 44807064.         5
 7 La Palma: Limited Series     25575000    6700000 20683065.         4
 8 Sirens: Limited Series       24533333.   3900000 25684691.         3
 9 Zero Day: Limited Series     24500000    2900000 32029986.         3
10 The Night Agent: Season 2    23966667.   5300000 25638708.         3
# ℹ 7,235 more rows
```

```r
consistent_high <- consistent_titles |>
  filter(
    n_periods >= 3,
    avg_views >= 20000000,
    min_views >= 10000000,
    sd_views <= 20000000
  ) |>
  arrange(desc(avg_views))

consistent_high
```

```
# A tibble: 2 × 5
  title                avg_views min_views sd_views n_periods
  <chr>                    <dbl>     <dbl>    <dbl>     <int>
1 Ms. Rachel: Season 1 45833333.  37300000 8093413.         3
2 Peppa Pig: Season 6  21233333.  16300000 3964677.         6
```

```r
consistent_high |>
  gt() |>
  cols_label(
    title = "Title",
    avg_views = "Average Views",
    min_views = "Minimum Views",
    sd_views = "Standard Deviation",
    n_periods = "Periods"
  ) |>
  fmt_number(
    columns = c(avg_views, min_views, sd_views),
    decimals = 0
  ) |>
  tab_header(
    title = md("**Titles with Consistently High Viewership**"),
    subtitle = "High average and minimum viewership with relatively low variation"
  )
```

Titles with Consistently High Viewership

High average and minimum viewership with relatively low variation

Title	Average Views	Minimum Views	Standard Deviation	Periods
Ms. Rachel: Season 1	45,833,333	37,300,000	8,093,413	3
Peppa Pig: Season 6	21,233,333	16,300,000	3,964,677	6

Question 4: Within each period, what fraction of time was spent on shows vs movies?

```r
#4.Within each period, what fraction of time was spent on ‘shows’ vs movies?

show_movie <- NETFLIX_DATA |>
  mutate(
    show = case_when(
      str_detect(title, "Season") ~ "Show",
      !is.na(runtime) & runtime > 240 ~ "Show",
      TRUE ~ "Movie"
    ) #distinguishing between show and movie (based on 4 hour rule)
  )

show_movie |>
  group_by(period, show) |>
  summarize(
    hours_viewed = sum(hours_viewed, na.rm = TRUE)
  ) |>
  group_by(period) |>
  mutate(
    fraction_time = hours_viewed / sum(hours_viewed)
  ) |>
  gt() |>
  cols_label(
    period = "Period",
    show = "Content Type",
    hours_viewed = "Hours Viewed",
    fraction_time = "Fraction of Time"
  ) |>
  fmt_number(
    columns = hours_viewed,
    decimals = 0
  ) |>
  fmt_percent(
    columns = fraction_time,
    decimals = 1
  ) |>
  tab_header(
    title = md("**Netflix Viewing Time: Shows vs. Movies**"),
    subtitle = "Titles classified using runtime and 'Season' heuristics"
  )
```

```
`summarise()` has regrouped the output.
ℹ Summaries were computed grouped by period and show.
ℹ Output is grouped by period.
ℹ Use `summarise(.groups = "drop_last")` to silence this message.
ℹ Use `summarise(.by = c(period, show))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

Netflix Viewing Time: Shows vs. Movies

Titles classified using runtime and 'Season' heuristics

Content Type	Hours Viewed	Fraction of Time
2023H1		
Movie	28,798,600,000	30.8%
Show	64,609,000,000	69.2%
2023H2		
Movie	2,739,300,000	4.2%
Show	62,191,700,000	95.8%
2024H1		
Movie	2,932,600,000	4.3%
Show	65,311,700,000	95.7%
2024H2		
Movie	2,413,500,000	3.5%
Show	65,766,400,000	96.5%
2025H1		
Movie	3,485,200,000	4.9%
Show	67,625,600,000	95.1%
2025H2		
Movie	3,741,000,000	5.2%
Show	67,642,300,000	94.8%
2026H1		
Movie	3,903,000,000	5.3%
Show	70,430,600,000	94.7%

Question 5: Live Events vs. Pre-Taped Content

skipped - was having trouble here

6. Corporate Strategy Memo

To: Netflix Leadership
From: Akshan Rai
Date: 10/18/26
Subject: Recommendations for 2027 Content Strategy

Dear Leadership,

The purpose of this memorandum is to develop a clear strategy moving into 2027 based off of the current data we have collected from the periods 2023 through 2026. The analytics team has had a few key findings that we believe present us with a strong opportunity looking ahead. The viewing data suggests that whilst our major blockbuster titles are important (Stranger Things, Squid Game, Wednesday) Netflix should balance the type of content it puts on the platform. In the first half of 2026, the top 10 titles accounted for 4.98 billion of the 74.33 billion total hours viewed, or roughly 6.7%. This indicates that since roughly 93% of views came from outside this range, we should not solely evaluate our success based on how the top-tier performs.

ⓘ **Note**

Another important finding was that some Netflix titles maintain strong viewership across different periods, as opposed to those that peak viewership solely around their release date. We found that the titles Ms. Rachel and Peppa Pig: Season 6 averaged 21 million and 45 million views over the course of six-periods. The recommendation from this finding is that Netflix should place more importance on titles like these that have a strong, consistent viewership rather than the seasonal blockbusters. We also evaluated catalog growth at Netflix. Between 2023 and 2026 the number of titles increased from 6,574 to 8,163. This indicates that a larger catalog provides more viewing-diversity to customers which has shown an increase of roughly 764,000 minutes of run-time. Continually expanding the catalog will work in congruence with our recommendation to balance investment into “blockbuster-heavy” titles.

A significant trend can be found when comparing viewing time between TV shows and movies. Although the data provided cannot directly identify the difference between TV shows and movies, we used the inference that a run-time over 4 hours (240 minutes) can be classified as a TV show (since most movies never reach that length). In 2023H1, movies accounted for 30.8% of total hours viewed and TV shows at 69.2%. The fascinating change comes in 2026H1 where movies dropped to only 5.3% and TV shows at 94.7%. This is a sharp decline for movie viewing-time, therefore Netflix needs to re-evaluate if investment efforts are sufficient for their TV-shows.

Overall, the data suggests that Netflix should pursue a more balanced content strategy moving into 2027. Although Blockbuster titles are efficient for generating significant viewing, they still account for only a small share of total viewing on Netflix, therefore investing in a diverse catalog will be of greater value.

ⓘ **AI Usage Statement**